AI Aware Cache and Prefetch Engine

Phase 1 technical report and separate context dependent choice research

The Phase 1 subsystem demonstrates how a small read-only cache and bounded prefetch engine can hide simulated memory latency on regular access patterns. Its evidence spans trace analysis, cache design, explicit prediction algorithms, functional SystemVerilog and measured policy comparisons.

The implemented design has a 1 KiB direct-mapped data cache, 16-byte lines and one outstanding memory transaction. Signed-stride and next-line predictors share the same controller. A separate CDM chapter reproduces the supplied notes arithmetic and proposes a future policy-selection connection. CDM is not implemented in the hardware.

The central result is conditional: stride helps long regular streams, while irregular or interleaved traffic can remove the benefit. Speculative cache pollution can cause a slowdown. These outcomes guide the next design change rather than being omitted.

Evidence	Verified baseline
Benchmark	13 traces, three policies, 39 runs
Returned data	32,637 RTL response checks
Current Python suite	38 tests including new starting models
Hardware evidence	Both generic synthesis policies pass CHECK
Date	1 November 2026

This is trace-driven subsystem simulation. No physical DRAM timing, real inference execution speedup, FPGA operation or competition outcome is claimed.

Design Thinking and Research Questions

Empathize with the compute-system designer: useful work stalls when required operands arrive late. Fetching more aggressively can also consume bandwidth or evict reusable data. The design problem therefore concerns timely useful delivery under limited resources.

Define success with paired completion cycles, demand latency, total memory transactions and prefetch fate accounting. A prediction can be accurate but late, or timely but harmful because it displaces a more useful line. Hit rate is one observation, not the objective by itself.

Ideate with a minimal cache-only reference, next-line prediction and repeated-delta stride prediction. Prototype a functional data path before adding adaptation. Test positive cases and adversarial cases with the same geometry, memory model and response-stall schedule.

Question	Evidence needed
Can prediction hide latency?	Regular stream off/on paired RTL
Does it waste traffic?	Demand plus speculative transactions
Can it damage reuse?	Hot-line pollution and conflict probes
Does it transfer to matrix accesses?	Filtered modelled A/B replay
Can CDM select a policy?	Separate future training and held-out evaluation

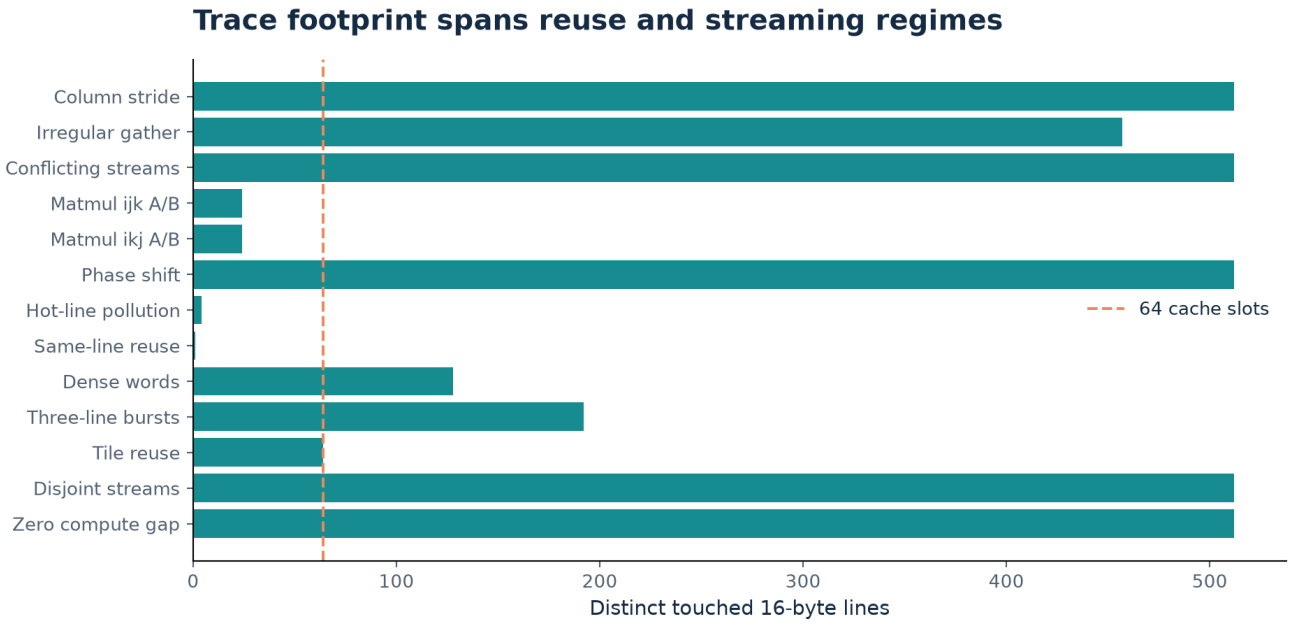
The five challenge levels are a progressive engineering structure. The provided slide names them but does not establish an official scoring rubric or acceptance guarantee.

Level 1 Memory Trace Evidence

Nitesh’s original feature-branch analysis and modelled matrix-multiplication generator remain part of the source history. The parser distinguishes address units and reports locality, line transitions and reuse gaps without pretending that they are cache hit measurements.

The 13 replay traces include regular, irregular, interleaved, phase-change, reuse, burst and pollution examples. The matrix bridge preserves only immutable input-operand A/B compute reads. It excludes initialization, all writes and mutable C. The numerical generator verifies matrix multiplication separately; the RTL checks address-derived backing words.

Archived Level 1 matrix analysis used 64-byte lines. The integrated replay reanalyses inputs at the actual 16-byte RTL geometry. Assigned model addresses and explicit compute gaps are modelling assumptions, not captured instruction timing.



Distinct touched lines at 16-byte geometry. The dashed line marks 64 cache slots; this is not a capacity requirement or a peak-live-data estimate.

Level 2 Cache Architecture

The baseline stores 64 lines of 128 data bits. Every slot also stores a full 28-bit line identity, a valid bit and a prefetched bit. Logical arrays occupy 10,112 bits, excluding predictor/controller/counter state. Full identities include redundant index bits and simplify inspection.

For a 32-bit byte address, line identity is bits 31:4, index is bits 9:4 by default and word selector is bits 3:2. Address 0x1234 selects line 0x123, index 35 and word 1. Addresses 0x1000 and 0x1400 conflict at index zero.

A direct-mapped refill overwrites the selected slot. There is no LRU choice when only one location exists. Phase 1 inserts speculative fills into the ordinary cache, deliberately retaining the demonstrated pollution failure for the Phase 2 comparison.

Decision	Implementation and rationale
Capacity	1 KiB data for a small inspectable baseline
Line size	16 bytes retains the verified refill interface
Associativity	Direct mapping simplifies lookup and verification
Writes	Unsupported under immutable read-only contract
Requests	One queued demand and one memory transaction
Software starting model	Executable untimed cache model with reference-count tests

Combinational array lookup and reset of every data entry favor a small register/distributed-memory design. Generic synthesis does not establish block-RAM inference, target timing or physical area.

Level 3 Prefetch Strategy

Both policies observe accepted demand addresses and ignore adjacent observations of the same line. Stride retains one last line, one signed delta and a saturating two-bit confidence. Two equal nonzero deltas qualify a candidate one stride ahead. Lines 100, 104 and 108 therefore propose 112.

Next-line can propose after the first distinct line, using current line plus one. This gives faster sequential startup and can waste traffic on larger strides. Both candidate generators use the same registered interface and controller.

The controller suppresses cached candidates and prioritizes pending/accepted demand at transaction selection. A selected SEND/WAIT prefetch is nonpreemptible. A demand for a line currently prefetched initially misses, then rechecks after fill, avoiding a duplicate memory transaction.

Policy decision	Phase 1 contract
Trigger	Accepted distinct-line demand, hit or miss
Distance and degree	One stride/line ahead, at most one candidate
Confidence	Repeated deltas; not learned usefulness probability
Bounds	Signed arithmetic with underflow/overflow rejection
Insertion	Ordinary direct-mapped fill
Known failures	Interleaved histories, bursts, late delivery and pollution

The executable Level 3 model and walkthrough notebook expose each state update. They explain predictor logic, not cycle timing or a performance improvement.

Level 4 Functional RTL and Verification

The IDLE state resolves queued demand or selects an eligible prediction. SEND holds its memory request and payload stable until ready. WAIT accepts a full line, updates data/identity/metadata and responds if this transaction serves demand.

CPU response validity and data remain stable through backpressure. A pending demand blocks later acceptance. A memory-fill edge also blocks acceptance, avoiding ambiguous simultaneous lookup/fill updates. Cache hits can proceed during speculative SEND/WAIT until a demand miss queues.

Shared synchronous reset flushes both cache and memory. The independent expected word is (byte address shifted right by two) XOR 0xcafe1234. Ordered scoreboards, boundary tests, arbitrary stalls, reset/disable tests and high-latency draining provide bounded correctness evidence.

Test family	Checked responses or result
Legacy workload and directed	3,673
Continuous request stress	2,000
Small-cache adversarial stress	10,000
High-latency and same-line smoke	20
Three-policy benchmark	16,944
Total RTL responses	32,637
Additional tests	38 Python tests plus predictor units

Both generic policy syntheses report zero CHECK problems. Tool-specific generic cell counts are 33,245 for stride and 32,877 for next-line. They are not FPGA LUT, power or timing sign-off results.

Level 5 Metrics and Fair Comparison

Every policy replays identical demand addresses, inter-demand compute gaps and a request-index-dependent 0–3-cycle consumer-stall schedule. Cache geometry and the deterministic memory service rule remain fixed. This is a serial demand driver, so gap zero does not mean a full-rate CPU.

Completion ends at consumption of the last demand response and excludes an unused final compute gap. Quiescence additionally drains pending predictor/candidate/controller work, requiring two settled quiet edges. These definitions replace the earlier package's fixed 100-cycle padding.

$$Demand\ \ hit\ \ rate\ = \frac{demand\ \ hits}{accepted\ \ demand\ \ accesses}$$

$$Speedup\ = \frac{baseline\ \ completion\ \ cycles}{policy\ \ completion\ \ cycles}$$

$$Consumed\ \ accuracy\ = \frac{timely\ \ consumption\ +\ \ late\ \ consumption}{issued\ \ prefetch\ \ transactions}$$

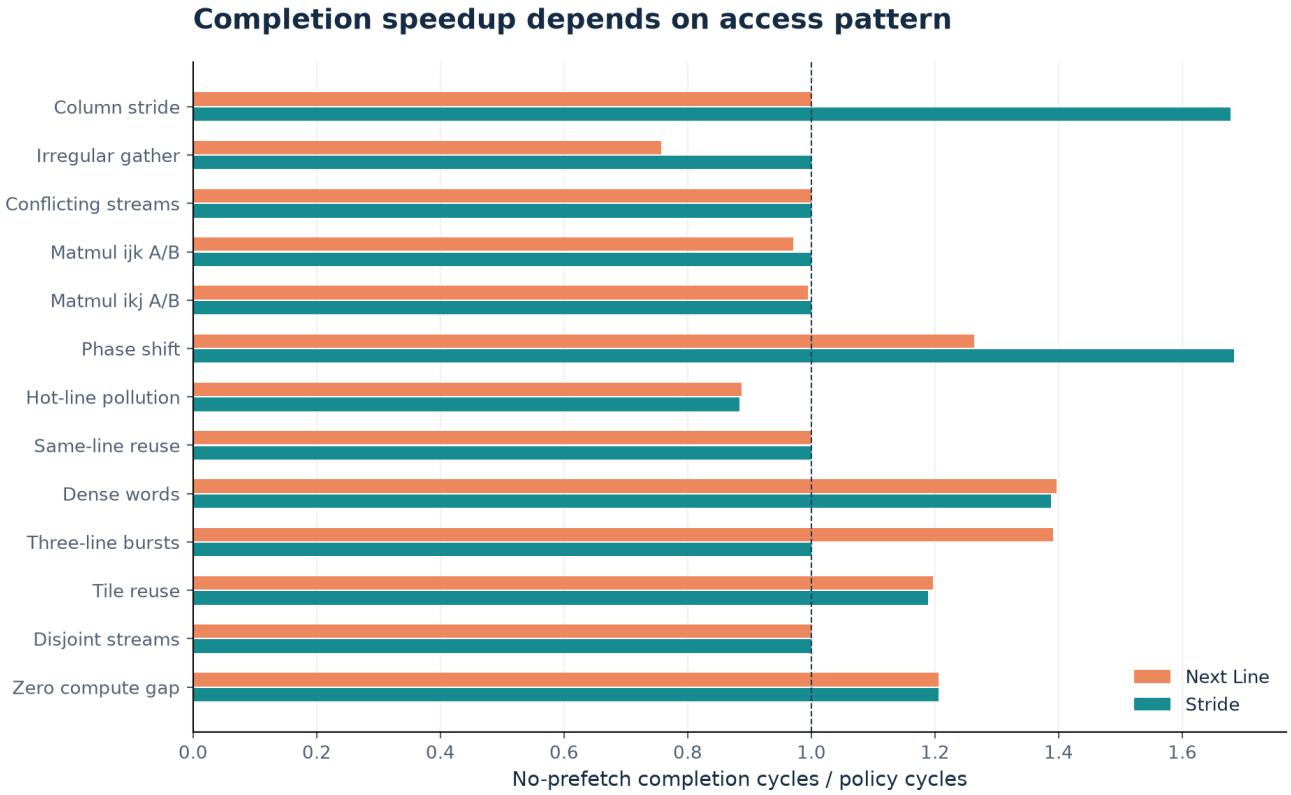
Reported quantity	Exact interpretation
Accepted latency	Acceptance to response availability
Offered latency	Offer to availability, including preaccept waiting
Traffic	All demand and prefetch line reads, 16 bytes each
Fates	Issued = timely + late + unused evicted + resident

Consumed accuracy is N/A when no prefetch issues. Resident lines are unresolved, and an unused eviction alone does not prove demand-cache pollution.

Completion Results and Their Causes

Stride recognizes the fixed line pitch in column_stride and achieves 1.678 times the no-prefetch completion speed. Dense word reads already obtain spatial hits, so higher hit rate produces a more modest completion gain. Reused tiles have fewer cold opportunities.

The fixed irregular gather seed generates no stride predictions. Interleaved streams repeatedly replace global history, removing the regularity visible in each individual operand stream. No universal gain follows from the good sequential results.

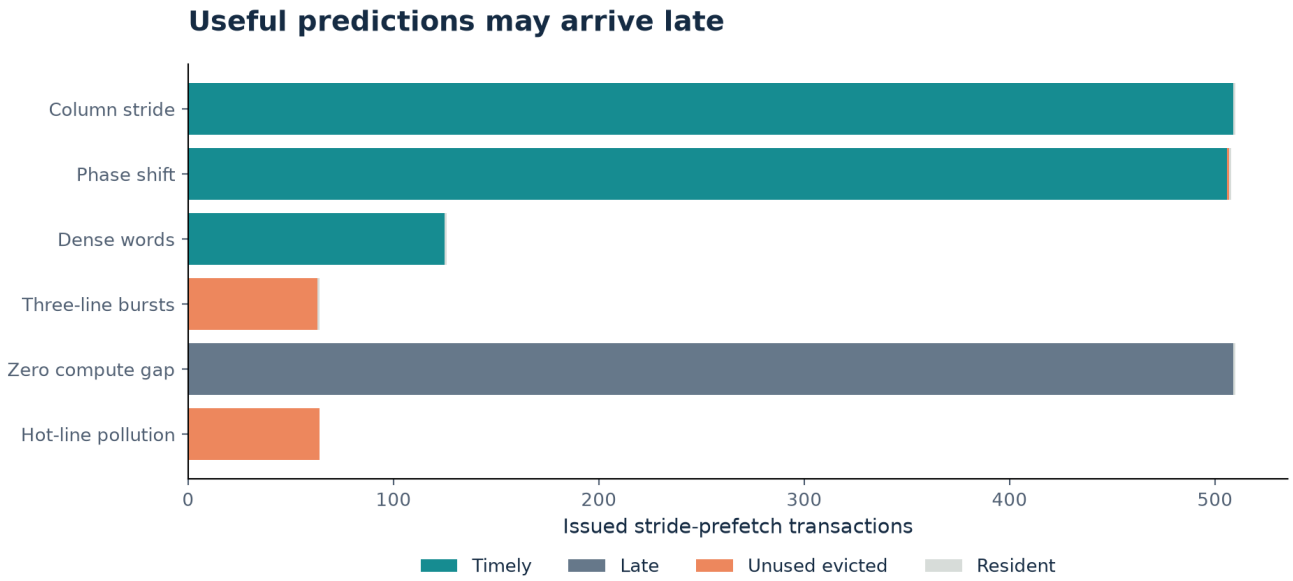


Completion-cycle ratios for all 13 traces. Values below one are slowdowns. The same recorded workload is compared across policies.

Latency Timeliness and Memory Traffic

The zero-gap stream has zero timely prefetch hits in either policy. Stride nevertheless starts speculative transfers before the next accepted demand and records 509 late consumptions. A useful late transfer can shorten a miss without changing its initial classification.

Total transactions must accompany latency and hit-rate plots. Next-line increases traffic sharply on column and disjoint-stream probes. Compute gaps sometimes hide those speculative transactions in elapsed time, but that is not a bandwidth saving.



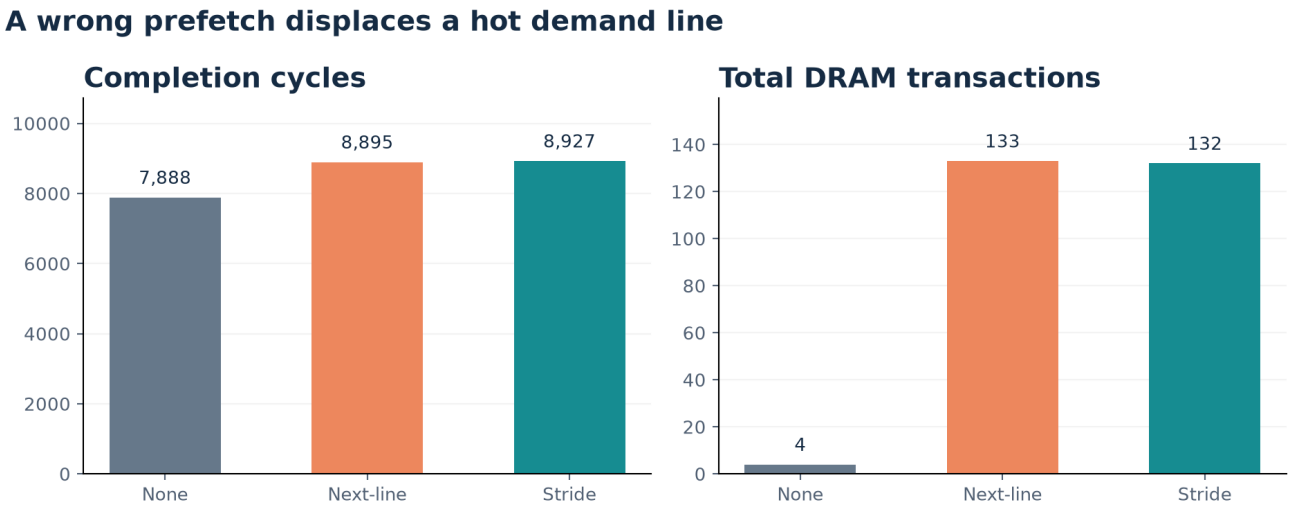
Stride-prefetch fates after quiescence. The pattern of timely, late, evicted and resident outcomes explains why hit rate alone is insufficient.

A Confirmed Pollution Failure

The pollution probe repeats the addresses 0x1000, 0x13d0, 0x13e0, 0x13f0 and 0x1000, 64 times, with a 20-cycle gap. Only four lines are demanded. A +1-line prediction from 0x13f0 fetches 0x1400, which maps to index zero and evicts hot 0x1000.

The cache-only run needs four memory transactions and 7,888 completion cycles. Stride needs 132 transactions and 8,927 cycles. The additional 64 demand misses and 64 unused predictions establish pollution in this paired experiment. All returned data remain correct.

A small separate prefetch buffer is the strongest next implementation target because it prevents speculative replacement of demand data. This is established stream-buffer engineering; the proposed contribution is a small verified integration and measured overhead.



Absolute transactions accompany the large traffic ratio because the baseline has only four cold reads.

Modelled Matrix Results and Limits

The ijk A/B subset completes in 6,940 cycles with no prefetch and with stride. Next-line takes 7,154 cycles while increasing memory transactions from 92 to 205. The ikj subset similarly shows no stride gain; next-line takes 6,443 versus 6,404 cycles and increases traffic from 61 to 126.

The individual row-major operands have regular access structure, but their interleaving can hide it from one global address history. This result motivates a multi-stream or policy-selection hypothesis, not a claim that current hardware accelerates a full matrix multiply or neural model.

The samples have explicitly assigned addresses and filtered traffic. Real application evidence needs a captured or instrumented workload with a documented address mapping and a defensible timing integration.

Subset	None	Next-line	Stride
ijk A/B cycles	6,940	7,154	6,940
ijk memory reads	92	205	92
ikj A/B cycles	6,404	6,443	6,404
ikj memory reads	61	126	61

Changing the cache to support mutable C, writes, concurrency or different line sizes would change the system contract and requires separate verification.

Low Rank CDM Mathematical Research

The supplied notes introduce a context-dependent choice model. For an available choice set S, each candidate's score sums interactions contributed by the other available alternatives. The low-rank factors represent target and context effects. This choice-set context is different from a workload's current cache/memory state.

The following equations use the notes' reparameterized model and exclude self-context. Rows of T and C hold the target and context vectors. The rank bound applies to the factor product M = TC transpose; replacing its diagonal by zero can change its rank.

$$u(x,z) = t(x)^T\ c(z), \quad x \neq z; \qquad M = T\ C^T$$

$$s(x,S) = \sum_{z \in S,\ z \neq x} u(x,z)$$

$$P(x \mid S;\ U) = \frac{\exp(s(x,S))}{\sum_{y \in S} \exp(s(y,S))}$$

$$Mean\ \ NLL = \frac{-1}{K} \sum_{k=1}^{K} \log\ P(y(k) \mid S(k))$$

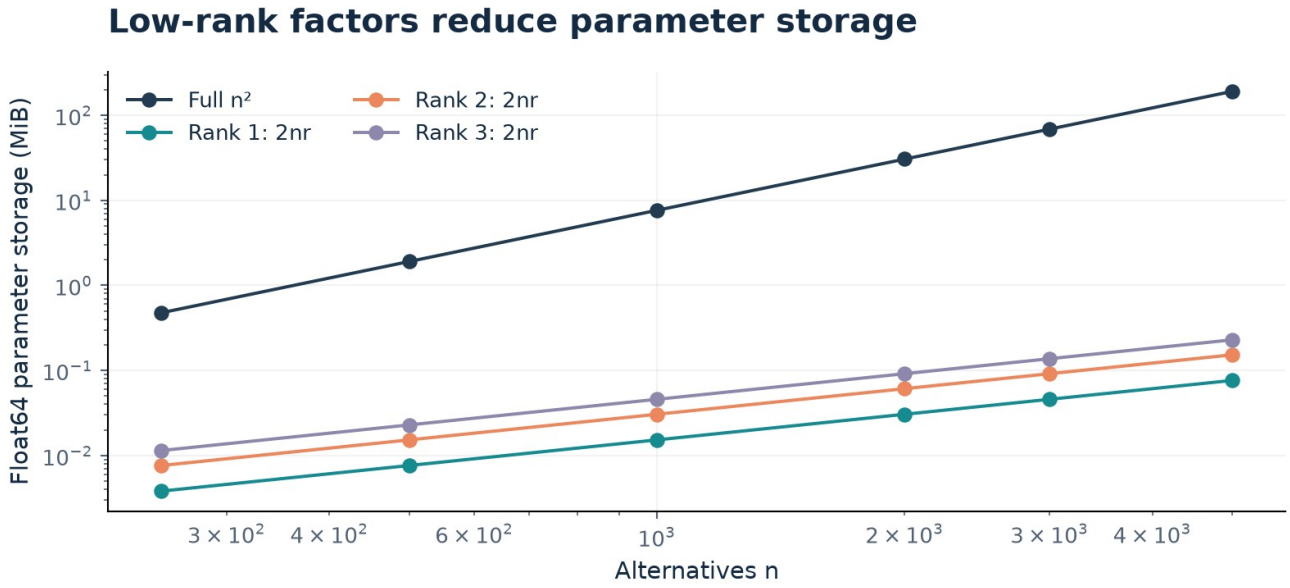
References: Shah, supplied Learning from Comparison Data draft, sections 4.4–4.6; Seshadri, Peysakhovich and Ugander, Discovering Context Effects from Raw Choice Data, ICML 2019. CDM is separate research and is not a fourth implemented RTL policy.

CDM Arithmetic and Storage Findings

The independent reference script reproduces the supplied six-observation illustration using ranks one, two and three. Mean NLL is 0.750970, 0.742501 and 0.753631 respectively. These are hand-chosen initial factors, not trained models. They do not establish rank-two superiority or held-out prediction quality.

The stored-entry counts below are analytic implementation quantities. Full identifiable degrees of freedom and factorization gauge ambiguities are different questions. Restricted choice-set data can leave further nonidentifiability.

$$N(full) = n^2; \quad N(factors) = 2nr; \quad MiB = \frac{8\ N}{2^{20}}$$



Float64 parameter-only storage. Gradients, optimizer state, data, temporary dense matrices and process RSS are excluded.

Proposed Context Policy Connection

A future selector could treat no prefetch, next-line and stride as a bounded legal alternative set. Standard CDM describes effects of which alternatives are available. Conditioning on miss rate, stride confidence or memory pressure requires an explicit feature-dependent extension; the supplied factors do not automatically encode those signals.

Generate labels from controlled policy experiments, split by workload/seed/phase and reserve held-out tests. Choose rank on validation data. Evaluate selector NLL, top-choice accuracy and calibration separately from actual completion-cycle speedup, traffic, timeliness, hardware state and timing. A selector that uses future trace information at runtime would be an invalid comparison.

Offline arithmetic may suggest a compact score calculation, but real hardware would require quantization, stable candidate identity, bounded compute and control integration. Argmax selection can avoid exponentials, yet this does not establish implementation cost or speedup.

Research step	Required gate
Define alternatives and context	Distinguish choice set from telemetry features
Construct data	Paired simulations, documented label objective
Train and validate	Separate workloads/seeds, declared rank selection
Compare	Fixed policies and simpler selector
Integrate later	Quantization, protocol and cost checks

The user-selected scope keeps this chapter a proposed research connection. No CDM cache performance or rerun of the notes' large PyTorch timing study is claimed.

Complete Benchmark Table

Completion cycles for all retained workloads. Detailed hit/miss, accepted/offered latency, prefetch fates, quiescence, transaction bytes and command/source provenance are in the accompanying CSV and per-run logs.

Workload	None	Next-line	Stride	Stride speedup
column stride	21,098	21,098	12,573	1.678x
embedding gather	12,757	16,849	12,757	1.000x
Conflicting streams	21,258	21,258	21,258	1.000x
Matmul ijk A/B	6,940	7,154	6,940	1.000x
Matmul ikj A/B	6,404	6,443	6,404	1.000x
phase shift	21,258	16,828	12,625	1.684x
pollution return	7,888	8,895	8,927	0.884x
same line	88	88	88	1.000x
tensor stream	7,554	5,410	5,445	1.387x
three line bursts	7,980	5,739	7,980	1.000x
tiled reuse	6,463	5,400	5,435	1.189x
Disjoint streams	21,417	21,417	21,417	1.000x
zero gap stream	11,197	9,284	9,289	1.205x

Source: level5/results/reference/summary.csv. The complete tables and publication plots are generated from the same measurements.

Discussion and Engineering Roadmap

Phase 1 is a complete runnable foundation for the stated five-level structure, with explicit remaining scope. It is address-history aware rather than tensor-semantic or CDM-driven hardware. Its strengths are functional data checking, fair policy wiring, outcome conservation and reproducible evidence.

The next implementation should isolate speculative lines in a small prefetch buffer, then add bounded consumption/waste feedback with recovery probes. Keep the original baseline and compare buffer-only versus buffer-plus-feedback. Region/PC tracking or a CDM selector should follow only if measurements justify their additional complexity.

Expand irregular seeds, gap/latency sweeps and true workload captures before making broad claims. Repeating a deterministic input establishes reproducibility, not statistical confidence. No finite simulation suite proves exhaustive correctness or a hackathon outcome.

Reference	Role
Nitesh Level 1 feature branch	Imported generator, notebook and data provenance
Jouppi 1990	Stream buffers and direct-map cache improvement
Chen and Baer 1995	Established hardware stride prefetching
Srinath et al 2007	Accuracy, lateness and pollution feedback
Seshadri et al 2019	CDM and low-rank choice modelling
Shah supplied draft 2026	Illustrative CDM factors and scaling discussion

Reproduction: make all; make notebooks; make figures with the optional plotting environment; make synth. MATLAB source is supplied and was not executed here. AI assistance is disclosed in repository provenance.